```
  void _upsertSubtitle(String type, Map<String, dynamic> json) {
    final text = (json['text'] as String? ?? '').trim();
    if (text.isEmpty) return;
    final bcp = (json['bcp47'] as String?) ?? _bcp47(json['language'] as String?);
    final id = json['utterance_id'] as String? ?? '';
    // With no id from the server, keep updating this side's open line instead of adding one
    // per partial result.
    var key = id.isEmpty ? '' : '$type:$id';
    if (key.isEmpty) {
      final open = transcripts.lastIndexWhere((t) => t.utteranceId.startsWith('$type#'));
      final reuse = open >= 0 && !transcripts[open].isFinal;
      key = reuse ? transcripts[open].utteranceId :
    '$type#${DateTime.now().microsecondsSinceEpoch}';
    }
    final line = TranscriptLine(
      utteranceId: key,
      fromPeer: type == 'subtitle_remote',
      sourceText: text,
      translatedText: '',
      srcLang: bcp,
      dstLang: bcp,
      isFinal: json['is_final'] == true,
    );
    final idx = transcripts.indexWhere((t) => t.utteranceId == line.utteranceId);
    if (idx >= 0) {
      transcripts[idx] = line;
    } else {
      transcripts.add(line);
      if (transcripts.length > 80) transcripts.removeAt(0);
    }
    notifyListeners();
  }

  String _bcp47(String? language) {
    final code = (language ?? '').toLowerCase();
    if (code.isEmpty) return src.bcp47;
    for (final item in supportedLanguages) {
      if (item.bcp47.toLowerCase() == code || item.iso == code) return item.bcp47;
    }
    return src.bcp47;
  }

  void _upsertTranscript(Map<String, dynamic> json) {
    final id = json['utterance_id'] as String? ?? '';
    final fromPeer = json['call_id'] != null &&
        (json['src_lang'] as String?) != session?.srcLang;
    final line = TranscriptLine(
      utteranceId: id,
      fromPeer: fromPeer,
      sourceText: json['source_text'] as String? ?? '',
      translatedText: json['translated_text'] as String? ?? '',
      srcLang: json['src_lang'] as String? ?? '',
      dstLang: json['dst_lang'] as String? ?? '',
      isFinal: json['is_final'] == true,
```

```
      latencyMs: (json['t_ms'] as Map?)?['e2e_server_ms'] as int?,
    );
    final idx = transcripts.indexWhere((t) => t.utteranceId == id && id.isNotEmpty);
    if (idx >= 0) {
      transcripts[idx] = line;
    } else {
      transcripts.add(line);
      if (transcripts.length > 80) transcripts.removeAt(0);
    }
    notifyListeners();
  }

  Future<void> _teardown() async {
    unawaited(callkit.endAll().catchError((_) {}));
    await _capture?.stop();
    await playback.stop();
    await _media?.close();
    await _webrtc?.hangup();
    await WakelockPlus.disable();
    _capture = null;
    _media = null;
    _webrtc = null;
    session?.phase = CallPhase.ended;
    session = null;
    transcripts.clear();
    connecting = false;
    _answering = false;
    _pendingRemoteSdp = null;
    _pendingIce.clear();
    _hearingRemote = false;
    notifyListeners();
  }

  void _syncMicGate() {
    _capture?.setDucked(
      session?.micMuted == true || session?.ducked == true || _hearingRemote,
    );
  }

  @override
  void dispose() {
    unawaited(_signaling?.dispose());
    super.dispose();
  }

  String _displayNameFor(String userId) {
    for (final c in directory) {
      if (c.userId == userId) return c.displayName;
    }
    return userId;
  }
}

===== mobile/lib/theme.dart =====
import 'package:flutter/material.dart';
```

```
import 'package:google_fonts/google_fonts.dart';

ThemeData buildTheme() {
  const bg = Color(0xFF07111F);
  const card = Color(0xFF0E1C30);
  const accent = Color(0xFF2EE6A6);
  const accent2 = Color(0xFF4C8DFF);
  final base = ThemeData.dark(useMaterial3: true);
  return base.copyWith(
    scaffoldBackgroundColor: bg,
    colorScheme: const ColorScheme.dark(
      primary: accent,
      secondary: accent2,
      surface: card,
    ),
    textTheme: GoogleFonts.dmSansTextTheme(base.textTheme).apply(
      bodyColor: Colors.white,
      displayColor: Colors.white,
    ),
    appBarTheme: const AppBarTheme(
      backgroundColor: Colors.transparent,
      elevation: 0,
      centerTitle: false,
    ),
  );
}

===== mobile/lib/widgets/call_controls.dart =====
import 'package:flutter/material.dart';

class CallControls extends StatelessWidget {
  const CallControls({
    super.key,
    required this.muted,
    required this.speakerOn,
    required this.onMute,
    required this.onSpeaker,
    required this.onLanguage,
    required this.onHangup,
  });

  final bool muted;
  final bool speakerOn;
  final VoidCallback onMute;
  final VoidCallback onSpeaker;
  final VoidCallback onLanguage;
  final VoidCallback onHangup;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.fromLTRB(16, 8, 16, 24),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceEvenly,
        children: [
```

```
          _Btn(icon: muted ? Icons.mic_off : Icons.mic, label: muted ? 'Unmute' : 'Mute', onTap:
    onMute),
          _Btn(
            icon: speakerOn ? Icons.volume_up : Icons.hearing,
            label: speakerOn ? 'Speaker' : 'Earpiece',
            onTap: onSpeaker,
          ),
          _Btn(icon: Icons.translate, label: 'Language', onTap: onLanguage),
          _Btn(
            icon: Icons.call_end,
            label: 'End',
            color: const Color(0xFFE23B4A),
            large: true,
            onTap: onHangup,
          ),
        ],
      ),
    );
  }
}

class _Btn extends StatelessWidget {
  const _Btn({
    required this.icon,
    required this.label,
    required this.onTap,
    this.color,
    this.large = false,
  });

  final IconData icon;
  final String label;
  final VoidCallback onTap;
  final Color? color;
  final bool large;

  @override
  Widget build(BuildContext context) {
    final size = large ? 68.0 : 54.0;
    return Column(
      children: [
        InkWell(
          onTap: onTap,
          customBorder: const CircleBorder(),
          child: Ink(
            width: size,
            height: size,
            decoration: BoxDecoration(
              shape: BoxShape.circle,
              color: color ?? const Color(0xFF173154),
            ),
            child: Icon(icon, color: Colors.white, size: large ? 30 : 22),
          ),
        ),
        const SizedBox(height: 8),
```

```
        Text(label, style: const TextStyle(fontSize: 11, color: Colors.white70)),
      ],
    );
  }
}

===== mobile/lib/widgets/language_card.dart =====
import 'package:flutter/material.dart';

import '../models/language.dart';

class LanguagePickerSheet extends StatelessWidget {
  const LanguagePickerSheet({
    super.key,
    required this.title,
    required this.selected,
    required this.onPick,
    this.exclude,
  });

  final String title;
  final Language selected;
  final Language? exclude;
  final ValueChanged<Language> onPick;

  @override
  Widget build(BuildContext context) {
    final height = MediaQuery.sizeOf(context).height * 0.78;
    return SafeArea(
      child: SizedBox(
        height: height,
        child: Container(
          decoration: const BoxDecoration(
            color: Color(0xFF0E1C30),
            borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
          ),
          child: Column(
            children: [
              const SizedBox(height: 10),
              Container(
                width: 42,
                height: 4,
                decoration: BoxDecoration(
                  color: Colors.white24,
                  borderRadius: BorderRadius.circular(99),
                ),
              ),
              Padding(
                padding: const EdgeInsets.fromLTRB(20, 16, 20, 8),
                child: Align(
                  alignment: Alignment.centerLeft,
                  child: Text(title, style: Theme.of(context).textTheme.titleLarge),
                ),
              ),
              Expanded(
```

```
                child: ListView.builder(
                  itemCount: supportedLanguages.length,
                  itemBuilder: (context, i) {
                    final lang = supportedLanguages[i];
                    final disabled = exclude != null && lang.bcp47 == exclude!.bcp47;
                    final active = lang.bcp47 == selected.bcp47;
                    return ListTile(
                      enabled: !disabled,
                      leading: Text(lang.flag, style: const TextStyle(fontSize: 22)),
                      title: Text(lang.englishName),
                      subtitle: Text(lang.nativeName),
                      trailing: active
                          ? const Icon(Icons.check_circle, color: Color(0xFF2EE6A6))
                          : null,
                      onTap: disabled
                          ? null
                          : () {
                              onPick(lang);
                              Navigator.pop(context);
                            },
                    );
                  },
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class LanguageCard extends StatelessWidget {
  const LanguageCard({
    super.key,
    required this.label,
    required this.language,
    required this.onTap,
  });

  final String label;
  final Language language;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return InkWell(
      onTap: onTap,
      borderRadius: BorderRadius.circular(20),
      child: Ink(
        padding: const EdgeInsets.all(16),
        decoration: BoxDecoration(
          color: const Color(0xFF0E1C30),
          borderRadius: BorderRadius.circular(20),
          border: Border.all(color: Colors.white10),
```

```
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(label, style: const TextStyle(color: Colors.white54, fontSize: 12)),
            const SizedBox(height: 10),
            Text(language.flag, style: const TextStyle(fontSize: 28)),
            const SizedBox(height: 8),
            Text(language.englishName, style: const TextStyle(fontWeight: FontWeight.w700, fontSize:
     18)),
            Text(language.nativeName, style: const TextStyle(color: Colors.white70)),
          ],
        ),
      ),
    );
  }
}

===== mobile/lib/widgets/transcript_bubble.dart =====
import 'package:flutter/material.dart';

import '../models/call_session.dart';
import '../models/language.dart';

String subtitleTurn(TranscriptLine line) {
  final who = line.fromPeer ? 'Caller' : 'You';
  final name = languageByCode(line.srcLang).englishName;
  return '$who ($name): ${line.sourceText}';
}

class TranscriptList extends StatefulWidget {
  const TranscriptList({super.key, required this.lines, this.emptyHint});

  final List<TranscriptLine> lines;
  final String? emptyHint;

  @override
  State<TranscriptList> createState() => _TranscriptListState();
}

class _TranscriptListState extends State<TranscriptList> {
  final _scroll = ScrollController();
  // The controller mutates one list in place, so the old widget's list always looks the
  // same as the new one. Compare against what was last seen here instead.
  int _seenCount = 0;
  String _seenLastText = '';
  bool _seenLastFinal = true;

  @override
  void initState() {
    super.initState();
    _remember();
    WidgetsBinding.instance.addPostFrameCallback((_) => _scrollToEnd(animate: false));
  }
```

```
  void _remember() {
    final lines = widget.lines;
    _seenCount = lines.length;
    _seenLastText = lines.isEmpty ? '' : lines.last.sourceText;
    _seenLastFinal = lines.isEmpty ? true : lines.last.isFinal;
  }

  @override
  void didUpdateWidget(TranscriptList oldWidget) {
    super.didUpdateWidget(oldWidget);
    final lines = widget.lines;
    final changed = lines.length != _seenCount ||
        (lines.isNotEmpty &&
            (lines.last.sourceText != _seenLastText || lines.last.isFinal != _seenLastFinal));
    _remember();
    if (changed) {
      WidgetsBinding.instance.addPostFrameCallback((_) => _scrollToEnd());
    }
  }

  void _scrollToEnd({bool animate = true}) {
    if (!_scroll.hasClients) return;
    final end = _scroll.position.maxScrollExtent;
    if (!animate) {
      _scroll.jumpTo(end);
      return;
    }
    _scroll.animateTo(
      end,
      duration: const Duration(milliseconds: 180),
      curve: Curves.easeOut,
    );
  }

  @override
  void dispose() {
    _scroll.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    if (widget.lines.isEmpty) {
      return Center(
        child: Text(
          widget.emptyHint ?? 'Listening\u2026\nSubtitles appear here as you speak.',
          textAlign: TextAlign.center,
          style: const TextStyle(color: Colors.white54),
        ),
      );
    }
    return ListView.builder(
      controller: _scroll,
      padding: const EdgeInsets.fromLTRB(16, 8, 16, 16),
      itemCount: widget.lines.length,
```

```
      itemBuilder: (context, i) => TranscriptBubble(line: widget.lines[i]),
    );
  }
}

class TranscriptBubble extends StatelessWidget {
  const TranscriptBubble({super.key, required this.line});

  final TranscriptLine line;

  @override
  Widget build(BuildContext context) {
    final align = line.fromPeer ? Alignment.centerLeft : Alignment.centerRight;
    return Align(
      alignment: align,
      child: Container(
        constraints: const BoxConstraints(maxWidth: 320),
        margin: const EdgeInsets.only(bottom: 10),
        padding: const EdgeInsets.fromLTRB(14, 10, 14, 10),
        decoration: BoxDecoration(
          color: line.fromPeer ? const Color(0xFF132844) : const Color(0xFF0F3D32),
          borderRadius: BorderRadius.circular(16),
          border: Border.all(
            color: line.isFinal ? Colors.transparent : const Color(0xFF2EE6A6),
          ),
        ),
        child: Text(
          subtitleTurn(line),
          style: TextStyle(
            fontSize: 16,
            height: 1.35,
            fontStyle: line.isFinal ? FontStyle.normal : FontStyle.italic,
            color: Colors.white.withValues(alpha: line.isFinal ? 1 : 0.85),
          ),
        ),
      ),
    );
  }
}

===== mobile/test/widget_test.dart =====
// This is a basic Flutter widget test.
//
// To perform an interaction with a widget in your test, use the WidgetTester
// utility in the flutter_test package. For example, you can send tap and scroll
// gestures. You can also use WidgetTester to find child widgets in the widget
// tree, read text, and verify that the values of widget properties are correct.

import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

import 'package:translate_language/main.dart';

void main() {
  testWidgets('Counter increments smoke test', (WidgetTester tester) async {
```

```
    // Build our app and trigger a frame.
    await tester.pumpWidget(const MyApp());

    // Verify that our counter starts at 0.
    expect(find.text('0'), findsOneWidget);
    expect(find.text('1'), findsNothing);

    // Tap the '+' icon and trigger a frame.
    await tester.tap(find.byIcon(Icons.add));
    await tester.pump();

    // Verify that our counter has incremented.
    expect(find.text('0'), findsNothing);
    expect(find.text('1'), findsOneWidget);
  });
}
```